

Implementation and Analysis of Fleury's, Hierholzer's, and Left-Hand Rule Algorithm for the CSES Mail Delivery Problem

Graph Theory — Week 3 Group Assignment



Ahmad Farras Favian Al Efasi - 5025251005

Daniel Pedrosaputra - 5025251171

Institut Teknologi Sepuluh Nopember

2026

1. PROBLEM DESCRIPTION

This assignment is modeled after the CSES Mail Delivery problem (<https://cses.fi/problemset/task/1691>). A mail carrier must start at the post office, travel along every street in the city exactly once to remain efficient, and return to the post office at the end of the route. This scenario is represented as a graph, where street crossings are modeled as vertices and roads connecting them are modeled as edges. Because the carrier must use every street exactly once and return to the starting point, this problem is equivalent to finding an Eulerian circuit in the graph.

For an Eulerian circuit to exist, the graph must satisfy two conditions: (1) every vertex with degree ≥ 1 must lie in the same connected component, and (2) every vertex must have even degree. CSES guarantees these conditions hold whenever a solution exists; if they do not hold, no valid route is possible and the problem expects an output indicating this ("IMPOSSIBLE").

Sample Input:

6 8

1 2

1 3

2 3

2 4

2 6

3 5

3 6

4 5

Sample Output:

1 2 6 3 2 4 5 3 1

2. ALGORITHM INTRODUCTION

a. Fleury's Algorithm

Fleury's Algorithm is a classical algorithm used to construct an Eulerian trail or Eulerian circuit in a graph — a traversal that uses every edge exactly once. It works by walking through the graph edge by edge, starting from a vertex with odd degree (if one exists) or any vertex (if the graph has an Eulerian circuit). At each step, the algorithm selects an unused edge to traverse, but follows one key rule: never cross a bridge (an edge whose removal would disconnect the remaining graph) unless there is no other option. This ensures the algorithm never strands itself in a disconnected portion of the graph before all edges have been visited. Because it must repeatedly test whether a candidate edge is a bridge, Fleury's Algorithm is straightforward to reason about but comparatively slow to execute on large graphs.

b. Hierholzer's Algorithm

Hierholzer's Algorithm is a more efficient algorithm for constructing an Eulerian trail or circuit. Instead of checking each edge for bridge status, it builds the traversal by forming closed cycles and splicing them together. The algorithm starts at a vertex and follows unused edges until it returns to that same vertex, forming a closed sub-cycle. If any vertex along this cycle still has unused edges, the algorithm starts a new cycle from that vertex, and once complete, merges it into the existing cycle. This process repeats until every edge in the graph has been used exactly once. Because it never needs to re-examine the graph structure to check for bridges, Hierholzer's Algorithm runs in linear time relative to the number of edges, making it significantly faster than Fleury's Algorithm on large graphs.

c. Left-Hand Rule Algorithm

The Left-Hand Rule, also known as the maze-following algorithm, is an alternative approach to constructing an Eulerian circuit, inspired by the classic "keep one hand on the wall" maze-solving technique. Rather than checking for bridges or building and splicing separate cycles, this method relies on a fixed rotational ordering of edges around each vertex (a rotation system). Upon arriving at a vertex through a given edge, the algorithm always proceeds along the next unused edge in that vertex's fixed order, cycling back to the beginning of the order if needed. Because the rotation order is set once and simply followed, this method produces a valid Eulerian circuit in graphs where every vertex has an even degree, without requiring bridge detection. It is used as the additional Eulerian algorithm required by the assignment, offering a more visual and intuitive alternative to Hierholzer's cycle-splicing strategy while retaining comparable efficiency.

3. ALGORITHM SIMULATION

a. Fleury's Algorithm

Fleury's Algorithm starts from vertex 1, since the graph has an Eulerian circuit. At every step, an unused edge is selected while avoiding a bridge whenever another unused edge is available. After an edge is traversed, it is removed from the graph. This process continues until all edges have been used.

For the sample graph, the traversal can be performed as follows:

Step	Current Vertex	Available Choice	Selected Edge	Reason
0	1	1–2, 1–3	—	Start at vertex 1
1	1	1–2, 1–3	1–2	1–2 is not a bridge
2	2	2–3, 2–4, 2–6	2–6	2–6 is not a bridge
3	6	6–3	6–3	Only unused edge
4	3	3–1, 3–2, 3–5	3–2	3–2 is not a bridge
5	2	2–4	2–4	Only unused edge
6	4	4–5	4–5	Only unused edge
7	5	5–3	5–3	Only unused edge
8	3	3–1	3–1	Only unused edge; returns to start

b. Hierholzer's Algorithm

Hierholzer's Algorithm constructs an Eulerian circuit by following unused edges until returning to the starting vertex. If unused edges remain, another cycle is created from a vertex already present in the current circuit and then spliced into it. This process continues until every edge has been used. This cycle-building and splicing approach is also how the course material describes the algorithm.

Starting from vertex 1, one possible traversal is:

Step 1 – Construct the initial cycle

Starting from 1:

$1 \rightarrow 2 \rightarrow 6 \rightarrow 3 \rightarrow 1$

This gives the cycle:

$1 \rightarrow 2 \rightarrow 6 \rightarrow 3 \rightarrow 1$

However, some edges remain unused:

2–3

2–4

3–5

4–5

Step 2 – Find a vertex with unused edges

Vertex 2 is already part of the current cycle and still has an unused edge 2–3.

Starting from vertex 2:

$2 \rightarrow 3 \rightarrow 5 \rightarrow 4 \rightarrow 2$

This produces another cycle:

$2 \rightarrow 3 \rightarrow 5 \rightarrow 4 \rightarrow 2$

Step 3 – Splice the cycles

The second cycle starts at vertex 2, which is already in the first cycle.

Original cycle:

$1 \rightarrow 2 \rightarrow 6 \rightarrow 3 \rightarrow 1$

Replace the occurrence of 2 with the second cycle:

$1 \rightarrow [2 \rightarrow 3 \rightarrow 5 \rightarrow 4 \rightarrow 2] \rightarrow 6 \rightarrow 3 \rightarrow 1$

Therefore:

$1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 4 \rightarrow 2 \rightarrow 6 \rightarrow 3 \rightarrow 1$

This is a valid Eulerian circuit because every edge is used exactly once.

Note: Hierholzer's algorithm can produce a different valid Eulerian circuit depending on which unused edge is selected first. The route does not have to be identical to the CSES sample output as long as every edge is used exactly once and the route starts and ends at 1.

c. Left-Hand Rule Algorithm

For the Left-Hand Rule, a fixed ordering of the edges around each vertex must first be established. The CSES problem gives the graph's edges but does not specify a rotation order around each vertex. Therefore, the following ordering is chosen for the simulation.

The edges are considered in the following local order:

Vertex	Fixed Edge Order
1	2, 3
2	6, 3, 4, 1
3	2, 5, 6, 1
4	2, 5
5	3, 4
6	2, 3

Starting at vertex 1, the algorithm follows the next unused edge according to this fixed ordering.

Starting at vertex 1, the algorithm follows the next unused edge according to this fixed ordering.

Step	Current Vertex	Selected Edge	Next Vertex
0	1	—	1
1	1	1–2	2
2	2	2–6	6
3	6	6–3	3
4	3	3–2	2
5	2	2–4	4
6	4	4–5	5
7	5	5–3	3
8	3	3–1	1

The resulting Eulerian circuit is:

$1 \rightarrow 2 \rightarrow 6 \rightarrow 3 \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow 3 \rightarrow 1$

All 8 edges are used exactly once, and the traversal returns to the starting vertex.

d. Summary

For the Left-Hand Rule, a fixed ordering of the edges around each vertex must first be established. The CSES problem gives the graph's edges but does not specify a rotation order around each vertex. Therefore, the following ordering is chosen for the simulation.

The edges are considered in the following local order:

Algorithm	Starting Vertex	Eulerian Circuit	Edges Used
Fleury's	1	$1 \rightarrow 2 \rightarrow 6 \rightarrow 3 \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow 3 \rightarrow 1$	8
Hierholzer's	1	$1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 4 \rightarrow 2 \rightarrow 6 \rightarrow 3 \rightarrow 1$	8
Left-Hand Rule	1	$1 \rightarrow 2 \rightarrow 6 \rightarrow 3 \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow 3 \rightarrow 1$	8

The three algorithms successfully construct Eulerian circuits for the sample graph. The resulting route does not need to be identical between algorithms; any route that uses every edge exactly once and returns to vertex 1 is a valid solution.

The distinction is in how the route is constructed: Fleury's avoids bridges, Hierholzer's builds and splices cycles, while the Left-Hand Rule follows a predetermined local edge ordering. The course material describes Fleury's as avoiding bridges and Hierholzer's as building and splicing cycles.

For your report, I recommend using this exact structure for Section 3. Then Section 4 can be the complexity/comparison, where we explain why Hierholzer's is more efficient than Fleury's and discuss the Left-Hand Rule.

4. ALGORITHM SIMULATION

The three algorithms use different approaches to construct an Eulerian circuit. Fleury's Algorithm repeatedly checks whether an edge is a bridge, while Hierholzer's Algorithm constructs cycles and combines them. The Left-Hand Rule follows a predetermined ordering of edges around each vertex.

a. Complexity Comparison

Algorithm	Main Approach	Time Complexity	Additional Space
Fleury's Algorithm	Select edges while avoiding bridges	$O(E^2)$	$O(V + E)$
Hierholzer's Algorithm	Build and splice Eulerian cycles	$O(E)$	$O(V + E)$
Left-Hand Rule	Follow a fixed edge ordering	$O(E)$	$O(V + E)$

Where V represents the number of vertices and E represents the number of edges.

b. Discussion

Fleury's Algorithm is relatively simple to understand because it directly applies the rule of avoiding bridges whenever possible. However, determining whether an edge is a bridge may require repeatedly examining the remaining graph. As a result, its typical implementation can require $O(E^2)$ time. The course material also lists Fleury's complexity as $O(E^2)$.

Hierholzer's Algorithm is more efficient because it does not repeatedly perform bridge detection. Instead, it follows unused edges and constructs cycles that are subsequently spliced together. This allows the algorithm to process the edges in $O(E)$ time. The course material gives Hierholzer's complexity as $O(E)$.

Left-Hand Rule uses a fixed ordering of edges around each vertex and follows that ordering while traversing unused edges. For this assignment's implementation, each edge is processed once, giving an $O(E)$ traversal time when the edge ordering is already available. However, unlike Fleury's and Hierholzer's algorithms, its correctness depends on having a suitable rotation ordering for the graph.

c. Overall Comparison

Criteria	Fleury's	Hierholzer's	Left-Hand Rule
Bridge detection	Required	Not required	Not required
Cycle construction	Direct traversal	Explicit cycle construction	Direct traversal
Time Complexity	$O(E^2)$	$O(E)$	$O(E)$
Approach	Avoid bridges	Build and splice cycles	Follow fixed edge order
Sample Result	Valid Eulerian circuit	Valid Eulerian circuit	Valid Eulerian circuit

For the sample test case, all three algorithms successfully produce a valid Eulerian circuit. The difference is mainly in their traversal strategy and computational complexity. For larger graphs, the linear-time approaches avoid the repeated bridge-testing work associated with the typical implementation of Fleury's Algorithm.

6. CONCLUSION

The three algorithms successfully produced a valid Eulerian circuit for the CSES Mail Delivery problem. Fleury's Algorithm finds the route by avoiding bridges whenever possible, but its repeated bridge checking results in a higher time complexity of $O(E^2)$. Hierholzer's Algorithm builds and combines smaller cycles and achieves $O(E)$ time complexity. The Left-Hand Rule follows a predetermined edge order and can also traverse the graph in $O(E)$ time.

Overall, the algorithms use different approaches to solve the same Eulerian circuit problem. Hierholzer's Algorithm provides an efficient solution because it processes the edges in linear time without requiring repeated bridge detection.

Tools: Claude, ChatGPT

Prompt:

1. Explain to me what I should do for the assignment (Refurbished W3 Eulerian Tour.pdf).
2. Give me an example of another algorithm that suits beside fleury's and hierholzer's.
3. Make a readme file for the repository based on this report.
4. help me make the c++ solution for the algorithms.